

Guide pratique

Virtualisation avec KVM / QEMU

Installation, configuration et gestion de machines virtuelles sous Linux

Rinaldis Damien

TP2 — Virtualisation

Introduction

Ce guide explique pas à pas comment installer KVM/QEMU sur une machine Linux Debian/Ubuntu, créer et configurer plusieurs machines virtuelles, puis y accéder à distance. Chaque étape est accompagnée des commandes exactes à saisir dans le terminal.

⚠ Toutes les commandes préfixées par `sudo` nécessitent les droits administrateur. Assurez-vous d'être connecté avec un compte ayant ces droits.

1

Vérifier que votre CPU supporte la virtualisation

Avant tout, il faut s'assurer que votre processeur est compatible avec la virtualisation matérielle (Intel VT-x ou AMD-V). Ouvrez un terminal et tapez :

```
egrep -c '^flags.*(vmx|svm)' /proc/cpuinfo
```

Résultat attendu : un nombre supérieur à 0. Si vous obtenez 0, la virtualisation est désactivée dans le BIOS — il faudra l'activer avant de continuer.

2

Mettre à jour le système

Avant d'installer de nouveaux paquets, mettez toujours le système à jour pour éviter les conflits de dépendances :

```
sudo apt update && sudo apt upgrade
```

Attendez la fin du processus. Si des mises à jour sont disponibles, confirmez avec Y quand c'est demandé.

3 Installer KVM, QEMU et les outils nécessaires

Installez tous les paquets requis en une seule commande. L'option `--no-install-recommends` évite d'installer des paquets inutiles comme une interface graphique lourde :

```
sudo apt install --no-install-recommends qemu-system libvirt-clients  
libvirt-daemon-system virtinst dnsmasq
```

```
root@tp:~# sudo apt update && sudo apt upgrade  
Hit:1 http://deb.debian.org/debian trixie InRelease  
Hit:2 http://security.debian.org/debian-security trixie-security InRelease  
Hit:3 http://deb.debian.org/debian trixie-updates InRelease  
All packages are up to date.  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0  
root@tp:~# sudo apt update && sudo apt upgrade  
Hit:1 http://security.debian.org/debian-security trixie-security InRelease  
Hit:2 http://deb.debian.org/debian trixie InRelease  
Hit:3 http://deb.debian.org/debian trixie-updates InRelease  
All packages are up to date.  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0  
root@tp:~#
```

Voici à quoi sert chaque paquet :

Paramètre	Valeur
qemu-system	Émulateur de machines virtuelles
libvirt-clients	Outils en ligne de commande (virsh)
libvirt-daemon-system	Service de gestion des VM
virtinst	Outil virt-install pour créer des VM
dnsmasq	Serveur DNS/DHCP pour le réseau des VM

Une fois installé, vérifiez que les modules KVM sont bien chargés dans le noyau :

```
lsmod | grep kvm
```

Résultat attendu : vous devez voir des lignes `kvm_intel` ou `kvm_amd` selon votre processeur.

4 Ajouter votre utilisateur aux bons groupes

Par défaut, seul root peut gérer les VM. Pour utiliser KVM et libvirt sans sudo à chaque fois, ajoutez votre utilisateur aux groupes `kvm` et `libvirt`. Remplacez `v` par votre nom d'utilisateur :

```
sudo adduser v kvm
```

```
sudo adduser v libvirt
```

```
root@tp:~# sudo adduser v libvirt
root@tp:~# sudo adduser v kvm
root@tp:~# █
```

⚠ Après cette étape, déconnectez-vous puis reconnectez-vous à votre session pour que les changements de groupe prennent effet.

5 Configurer le réseau virtuel

Libvirt crée automatiquement un réseau virtuel NAT appelé « default ». Ce réseau permet à vos VM d'accéder à Internet via votre machine hôte. Vérifiez d'abord son état :

```
sudo virsh net-list --all
```

```
root@tp:~# sudo virsh net-list --all
Name          State      Autostart    Persistent
-----
default       inactive  no           yes

root@tp:~# █
```

Si le réseau est marqué comme inactif, démarrez-le et activez son démarrage automatique :

```
sudo virsh net-start default
sudo virsh net-autostart default
```

```
root@tp:~# sudo virsh net-autostart default
Network default marked as autostarted
```

Résultat attendu : le réseau default doit apparaître avec l'état actif et autostart activé.

6 Créer la première VM — vm-passerelle (Debian)

Cette VM sera créée à partir d'un ISO d'installation Debian. La commande virt-install prend en charge tout le processus :

```
sudo apt install qemu-utils

virt-install --name vm-passerelle --memory 2048 --disk size=12 --os-variant
debian10 --cdrom /home/v/Downloads/debian-13.3.0-amd64-netinst.iso

WARNING KVM acceleration not available, using 'qemu'
WARNING Failed to detect osinfo OS name from install media, using fallback name 'linux2024'.
Please file a bug against virt-install if you expected the detection to succeed.
WARNING The requested volume capacity will exceed the available pool space when the volume is fully allocated. (12
288 M requested capacity > 2269 M available)
WARNING Requested memory 2048 MiB is less than the recommended 3072 MiB for OS linux2024
WARNING Unable to connect to graphical console: virt-viewer not installed. Please install the 'virt-viewer' packag
...
WARNING No console to launch for the guest, defaulting to --wait -1

starting install...
Allocating 'vm-passerelle.qcow2' | 12 GB 00:00
creating domain... | 00:00

Domain is still running. Installation may be in progress.
Waiting for the installation to complete.
```

Paramètre	Valeur
--name	Nom de la VM : vm-passerelle
--memory 2048	2 Go de RAM
--disk size=12	Disque virtuel de 12 Go (créé automatiquement)
--os-variant	Optimisation pour Debian 10
--cdrom	Chemin vers l'ISO d'installation

Une fenêtre s'ouvre avec l'installateur Debian. Suivez les étapes d'installation normalement. Choisissez d'installer SSH server lors de la sélection des logiciels.

7

Créer la deuxième VM — vm-texte (Debian 11 via virt-builder)

virt-builder génère directement une image disque prête à l'emploi, sans passer par un installateur interactif. C'est beaucoup plus rapide. Commencez par installer les outils nécessaires :

```
sudo apt install libguestfs-tools
```

```
v@tp:~$ sudo apt install libguestfs-tools
Installing:
libguestfs-tools

Installing dependencies:
attr                    libintl-xs-perl        lvm2
augeas-lenses          liblfflib0t64          lzop
bash-completion        libaugeas0             mdadm
binutils               libbfi1               liblvm2cmd2.03         mttools
binutils-common        libbinutils            libnetpbm11t64        netpbm
binutils-x86-64-linux-gnu libconfig1             libreisertfscore0t64  reisertfsprogs
btrfs-progs            libctf-nobfd0          librpm-sequoia-1       rpm-common
cryptsetup-bin         libctf0                librpm10               scrub
curl                   libdate-manip-perl     librpmio10            sleuthkit
db-util                libdevmapper-event1.02.1 libstrace               squashfs-tools
dbus-1-util            libdw1                 libstring-shellquote-perl syslinux
dmeventd               libfuse2t64            libsys-virt-perl       syslinux-common
extlinux               libguestfs-hfsplus     libtsk19t64           thin-provisioning-tools
f2fs-tools             libguestfs-perl        liburcu8t64           util-linux-extra
gawk                   libguestfs-reiserfs    libvhdl1              uuid-runtime
gdisk                  libguestfs-xfs          libvmdk1              virt-p2v
guestfish              libguestfs-0t64        libwin-hivex-perl     xfsprogs
guestfs-tools          libhfs0t64             libxml-xpath-perl     zerofree
guestmount             libhivex0              libyara10
hfsplus                libintl-perl           lsscsi
icoutils               libintl1               lsscsi

Suggested packages:
augeas-doc      gawk-doc      libguestfs-jfs      default-mta      xfsdump
binutils-doc   libterm-readline-gnu-perl  libguestfs-nilfs  | mail-transport-agent  quota
gprofng-gui    | libterm-readline-perl-perl  libguestfs-rescue  | floppyd
binutils-gold  augeas-tools  libguestfs-rsync   autopsy
duperemove     libguestfs-gfs2  libguestfs-zfs     mac-robber
```

Générez l'image disque Debian 11 de 10 Go :

```
virt-builder debian-11 --size 10G -o vm-texte.qcow2

v@tp:~$ virt-builder debian-11 --size 10G -o vm-texte.qcow2
[ 1.9] Downloading: http://builder.libguestfs.org/debian-11.xz
#####
```

Créez la VM à partir de l'image générée :

```
virt-install --name vm-texte --memory 4096 --disk vm-texte.qcow2
```

Une fois la VM démarrée, testez-la :

- Connectez-vous en SSH
- Faites une mise à jour des paquets : `sudo apt update && sudo apt upgrade`
- Testez les commandes guestfs sur l'image : `virt-ls`, `virt-cat`, `virt-edit`

8 Créer la troisième VM — vm-graphique (Ubuntu via qemu-img)

Cette méthode crée d'abord un disque vide, puis installe l'OS depuis un ISO. C'est la méthode la plus manuelle et la plus flexible.

Créez le disque virtuel au format qcow2 (format recommandé pour KVM) :

```
qemu-img create -f qcow2 vm-graphique.qcow2 10G
```

Lancez ensuite l'installation depuis l'ISO Ubuntu en pointant vers le disque créé. Utilisez `--cpu host` pour de meilleures performances :

Paramètre	Valeur
Nom	vm-graphique

RAM	4 Go
Disque	vm-graphique.qcow2 (10 Go)
CPU	host — utilise les extensions du processeur physique
OS installé	Xubuntu (installation minimale)

9 Gérer vos machines virtuelles avec virsh

La commande virsh permet de contrôler toutes vos VM depuis le terminal. Voici les commandes essentielles :

```
virsh list --all      # Voir toutes les VM (actives et arrêtées)
virsh start vm-texte  # Démarrer une VM
virsh shutdown vm-texte # Arrêt propre (comme éteindre un vrai PC)
virsh destroy vm-texte # Arrêt forcé (si la VM ne répond plus)
virsh suspend vm-texte # Mettre en pause
virsh resume vm-texte  # Reprendre depuis la pause
```

10 Configurer le réseau avancé (Bridge et NAT)

Mode Bridge

Le mode bridge connecte la VM directement à votre réseau local, comme si c'était une vraie machine branchée sur le même switch. Installez d'abord bridge-utils :

```
sudo apt install bridge-utils
```

Modifiez ensuite le fichier /etc/network/interfaces pour créer un bridge br0 associé à votre interface réseau physique (ex: eth0). La VM utilisera alors une adresse IP du même sous-réseau que votre réseau local.

Mode NAT (par défaut)

Le réseau virbr0 créé automatiquement par libvirt fonctionne en NAT. Les VM ont des adresses dans le sous-réseau 192.168.122.0/24 et accèdent à Internet via votre machine hôte. Pour se connecter en SSH depuis l'hôte vers une VM :

```
ssh utilisateur@192.168.122.X # Remplacez X par l'IP de la VM
```

11 Accès graphique à distance : SPICE et VNC

Deux protocoles permettent d'afficher l'interface graphique d'une VM à distance :

Protocole	Avantages	Quand l'utiliser
SPICE	Rapide, audio, copier-coller, redimensionnement auto	VM locales avec interface graphique

VNC	Universel, compatible tous OS et clients	Accès distant depuis n'importe quel OS
-----	--	--

Pour activer SPICE ou VNC, éditez le fichier XML de la VM avec virsh :

```
sudo virsh edit vm-graphique
```

Ajoutez ou modifiez la section <graphics> dans le XML pour y configurer le protocole choisi, le port et l'adresse d'écoute.

⚠ Utilisez SPICE en priorité si vous êtes sur le même réseau local — c'est nettement plus fluide que VNC pour une interface graphique.